
RAPPORT DE PROJET

Projet Création Numérique : TinderFood

Auteurs :

Alexandre

Victor

Vadim

Galatée

BATISSE

ROULEAU

DZIERZBICKI

PRIÉ

Enseignant :

Mr. BONNET

Nous attestons que ce travail est original, qu'il est le fruit d'un travail commun et qu'il a été rédigé de manière autonome.

Paris, le 23/04/2025

Table des matières

I. Introduction	3
A. Contexte	3
B. Problématique	3
C. Objectif	4
II. L'équipe	4
A. Présentation de l'équipe	4
B. Organisation de l'équipe et diagramme de Gantt	5
III. Développement	6
A. Architecture Technique Globale	6
B. Back-end et Gestion des Données	8
C. Front-end et Interface Mobile	12
D. Défis Techniques et Solutions	15
IV. Conclusion	16
A. Perspectives	16
B. Intérêt professionnel	17
C. Bilan sur le travail d'équipe	18

I. Introduction

A. Contexte

Choisir un restaurant s'effectue aujourd'hui au croisement de multiples sources : cartes (Google Maps, TheFork, TripAdvisor), plateformes de livraison qui proposent aussi des parcours "découverte", et réseaux sociaux où l'on consomme des recommandations sous forme de vidéos courtes et de carrousels. Sur mobile, ces services privilégient généralement des listes longues, des cartes denses et des filtres multiples (cuisine, note, distance, budget), avec un passage constant entre écrans pour consulter photos, menus ou itinéraires.

Dans cette culture de l'abondance, le parcours réel de décision se fait souvent "à la volée" : on ouvre une application, on saisit un type de cuisine, on parcourt quelques visuels et extraits de menus, puis l'on revient à la carte ou à la liste pour comparer un autre établissement. Les choix sont fortement influencés par le contexte immédiat (heure, localisation, compagnie) et par des signaux simples (photo appétissante, proximité, note globale).

C'est dans ce paysage que s'inscrit TinderFood : une application mobile qui met en scène la comparaison deux à deux. L'utilisateur, après avoir indiqué son type de cuisine, voit s'afficher deux restaurants simultanément ; une flèche ouvre une fiche synthétique (photos, aperçu du menu), un clic permet de conserver son favori, et un nouvel opposant apparaît. À tout moment, un bouton « J'ai choisi mon restaurant » clôt la recherche ; l'itinéraire est proposé via un lien Maps et un système d'avis internes complète l'expérience.

B. Problématique

Aujourd'hui, l'offre est immense et dispersée entre applis d'avis, plateformes de livraison et réseaux sociaux. Au moment de choisir un restaurant, l'utilisateur se retrouve vite indécis. Les interfaces dominantes favorisent l'exploration mais ralentissent la décision sur mobile, surtout quand il faut aller vite (pause déjeuner, sortie le soir). À cela s'ajoutent des signaux difficiles à comparer (notes, photos, promos, distance) et des infos parfois incomplètes (menu, horaires, fourchette de prix), qui entretiennent le doute.

Cette surcharge d'informations crée une fatigue de comparaison (allers-retours entre fiches, vérifications répétées) et complique la décision à plusieurs : chacun a des préférences implicites (cuisine, budget, temps de trajet) rarement posées clairement. Beaucoup finissent par repousser le choix ou par opter pour un lieu "par défaut", sans réelle conviction. La problématique est donc de transformer ce parcours morcelé et coûteux en attention en un processus de décision court, fluide et fiable, adapté à l'usage mobile et aux contraintes réelles de la situation.

C. Objectif

Les objectifs fixés sont dès lors les suivants : Livrer une application mobile simple et efficace qui aide l'utilisateur à décider vite où manger. Concrètement, proposer un parcours de duels (deux restaurants à la fois), une fiche info courte (photos, menu), un arrêt clair ("j'ai choisi mon restaurant") et l'itinéraire immédiat via Maps, le tout avec une expérience fluide et des gestes naturels.

Côté technique, assurer une chaîne complète et fiable (authentification, base AWS, API, affichage mobile), des données propres (restaurants, avis internes) et une navigation réactive. Pour mesurer la réussite du MVP : un temps médian jusqu'au choix < 90 s, 5 à 8 duels en moyenne avant décision, aucun crash bloquant en démo, animations fluides et retours utilisateurs ≥ 4/5 sur les restaurants "matchés".

II. L'équipe

A. Présentation de l'équipe

Vadim - Majeur Systèmes embarqués		Compétences : développement mobile (React Native/Expo), interactions tactiles et animations fluides, gestion de la localisation et des liens cartographiques, optimisation des performances graphiques, mise au point et packaging des versions.
Galatée - Majeur Énergie & Environnement		Compétences : Conception UX/UI, rédaction des interfaces, accessibilité (bon contraste, tailles, navigation), organisation des tests utilisateurs et analyse des retours, documentation produit.

Victor - Majeur Data & IA		Compétences : Modélisation et nettoyage des données (SQL), indicateurs d'usage et analyses (temps jusqu'au choix, duels par session), méthodes de recommandation (notation type Elo, amorce d'apprentissage automatique), contrôle qualité des jeux de données.
Alexandre - Majeur Énergie & Environnement		Compétences : Conception et optimisation de la base MySQL (schéma, index), développement de l'API (Node/Express), sécurité applicative de base (validation, CORS, limitation de débit), déploiement et supervision sur AWS.

B. Organisation de l'équipe et diagramme de Gantt

Pour l'organisation des tâches de ce projet nous avons utilisé la méthode MOSCOW ce qui nous a permis de prioriser ainsi que d'ajuster notre diagramme de Gantt :

M	Must Have Duels "gagnant vs challenger", connexion utilisateur, BDD AWS, swipes & likes, bouton "J'ai choisi".
S	Should Have Filtres avancés, localisation précise, avis/notes, lien Maps.
C	Could Have Swipe natif, statistiques, algorithme intelligent de proposition.
W	Won't Have App publiée, paiement intégré, réseau social complet.

Figure [1] : Analyse Moscow

S1	04-10/11	Cadrage & Architecture BDD	Tous / Alex & Victor
S2	11-17/11	BDD AWS, Back-end	Alex / Vadim & Galatée
S3	18-24/11	Cœur du duel, Html and CSS form	Vadim & Galatée / Alex
S4	25/11-01/12	Filtres, API & Dataset	Galatée / Alex / Victor
S5	02-08/12	Localisation & Profil	Victor & Alex / Vadim
S6	09-17/12	Tests, Corrections & Démo	Tous

Figure [2] : Diagramme de gantt et répartition des tâches

III. Développement

A. Architecture Technique Globale

1. Vue d'ensemble (Client-Serveur) :

L'architecture technique de TinderFood repose sur un modèle Client-Serveur découplé, structuré en trois tiers distincts. Cette organisation est essentielle pour garantir la fluidité de l'expérience utilisateur sur mobile tout en assurant la sécurité et la centralisation des données.

Le système se compose des entités suivantes :

- Le Client (Front-end) : Il s'agit de l'application mobile installée sur le terminal de l'utilisateur. Développée pour offrir une réactivité maximale (animations des

duels, navigation tactile), elle est chargée de la présentation visuelle, de la capture des interactions (clics, swipes) et de la récupération des coordonnées GPS locales. Le client est "léger" : il ne stocke pas la totalité de la base de données, mais interroge le serveur au besoin.

- Le Serveur (Back-end) : C'est le chef d'orchestre de l'application. Hébergé à distance, il expose une API REST (Application Programming Interface) qui reçoit les requêtes du client. Il contient la logique métier : validation des utilisateurs, algorithme de sélection des restaurants à opposer, et enregistrement des votes.
- La Communication : Les échanges entre le mobile et le serveur s'effectuent via le protocole HTTP standard. Les données transitent au format JSON, un format léger idéal pour les réseaux mobiles, permettant de transférer rapidement les informations des restaurants (noms, photos, distances) et les actions de l'utilisateur (connexion, choix du restaurant).

Cette architecture permet une séparation claire des responsabilités : l'équipe mobile (Vadim, Galatée) a pu se concentrer sur l'interface et l'expérience utilisateur, tandis que l'équipe Back-end (Alexandre, Victor) a structuré la logique de données et l'infrastructure, sans que les modifications d'une partie n'impactent négativement l'autre.

2. Infrastructure Cloud (AWS) :

Pour l'hébergement de notre base de données relationnelle, nous avons pris la décision stratégique de migrer notre environnement de développement local vers une infrastructure Cloud robuste fournie par Amazon Web Services (AWS). Concrètement, nous exploitons le service Amazon RDS (Relational Database Service) pour maintenir notre instance MySQL en ligne de manière continue.

Ce choix répond premièrement à un impératif de centralisation et de disponibilité. Contrairement à une base de données hébergée localement qui restreindrait l'accès au seul poste du développeur, le cloud rend la donnée accessible simultanément à l'ensemble de l'équipe technique, qu'il s'agisse du développement mobile ou du backend. Cela assure une source de vérité unique et garantit que le service reste opérationnel vingt-quatre heures sur vingt-quatre pour les utilisateurs finaux, indépendamment de l'état de nos machines personnelles. C'est la principale raison pour laquelle nous avons décidé d'héberger notre base de données sur AWS.

Par ailleurs, cette architecture assure la scalabilité du projet en permettant d'ajuster les ressources du serveur selon le trafic sans interruption de service. Enfin, la sécurité est renforcée par l'utilisation de variables d'environnement, protégeant ainsi

les clés d'accès à la base de données et évitant toute exposition de données sensibles dans le code source.

Voici un schéma qui résume l'architecture globale de notre application :

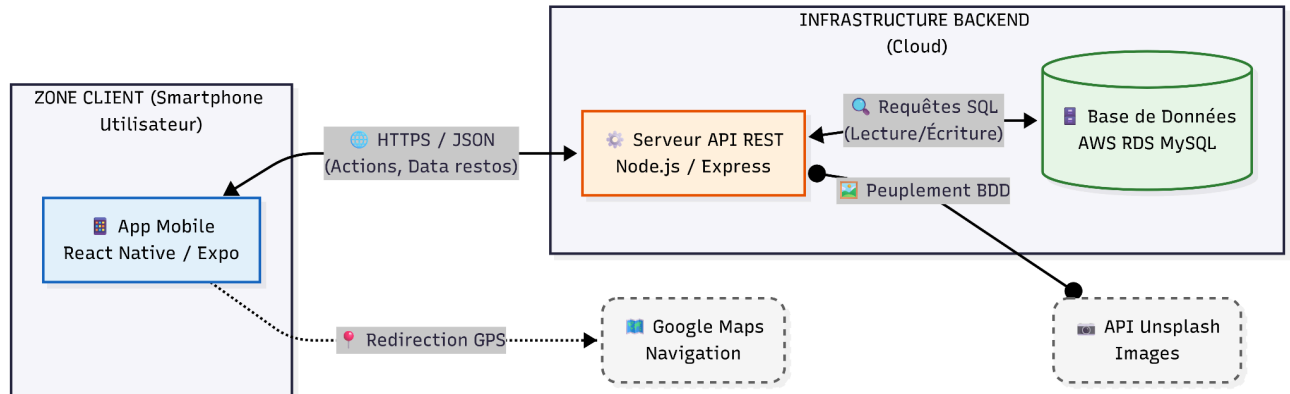


Figure [3] : Architecture Globale de l'Application

B. Back-end et Gestion des Données

1. Modélisation de la Base de Données (MySQL) :

La robustesse de TinderFood repose sur un schéma relationnel MySQL rigoureux pour garantir l'intégrité des données et la rapidité des requêtes, même lors de multiples accès simultanés.

La structure s'articule autour de cinq tables interconnectées, hébergées sur notre instance AWS RDS :

- **Users (Utilisateurs) :** Cette table centrale stocke les informations de compte (email, adresse) et sécurise l'accès via le stockage du password_hash (généré par Bcrypt), plutôt que le mot de passe en clair. Elle enregistre également la dernière position connue (latitude, longitude) pour faciliter le tri géolocalisé lors des futures sessions.
- **Restaurants :** Il s'agit du catalogue principal. Outre les informations statiques (Nom, Type de cuisine, Siret), cette table a été enrichie de champs dynamiques comme photo_url (peuplé via API) et des indicateurs booléens (Vegetarian, Delivery) pour alimenter les filtres de recherche.
- **Swipes (Interactions de jeu) :** Cette table enregistre chaque duel joué. Elle lie un user_id à un restaurant_id avec un horodatage (swipe_time). C'est une table clé pour l'algorithme de recommandation futur, car elle permet de comprendre quels restaurants ont "gagné" face à d'autres.

- Likes (Favoris) : Distincte des swipes, cette table gère la persistance des restaurants que l'utilisateur souhaite retrouver plus tard. Elle utilise une contrainte d'unicité pour éviter les doublons dans le profil utilisateur.
- Reviews (Avis) : Elle permet le stockage des notes (1 à 5 étoiles) et commentaires textuels, créant ainsi un écosystème d'avis propre à l'application, indépendant des plateformes externes.

Voici ci-dessous le diagramme Entité-Association (ERD) de la base de données TinderFood, détaillant les relations entre les utilisateurs, les restaurants et les interactions (swipes, likes, avis).

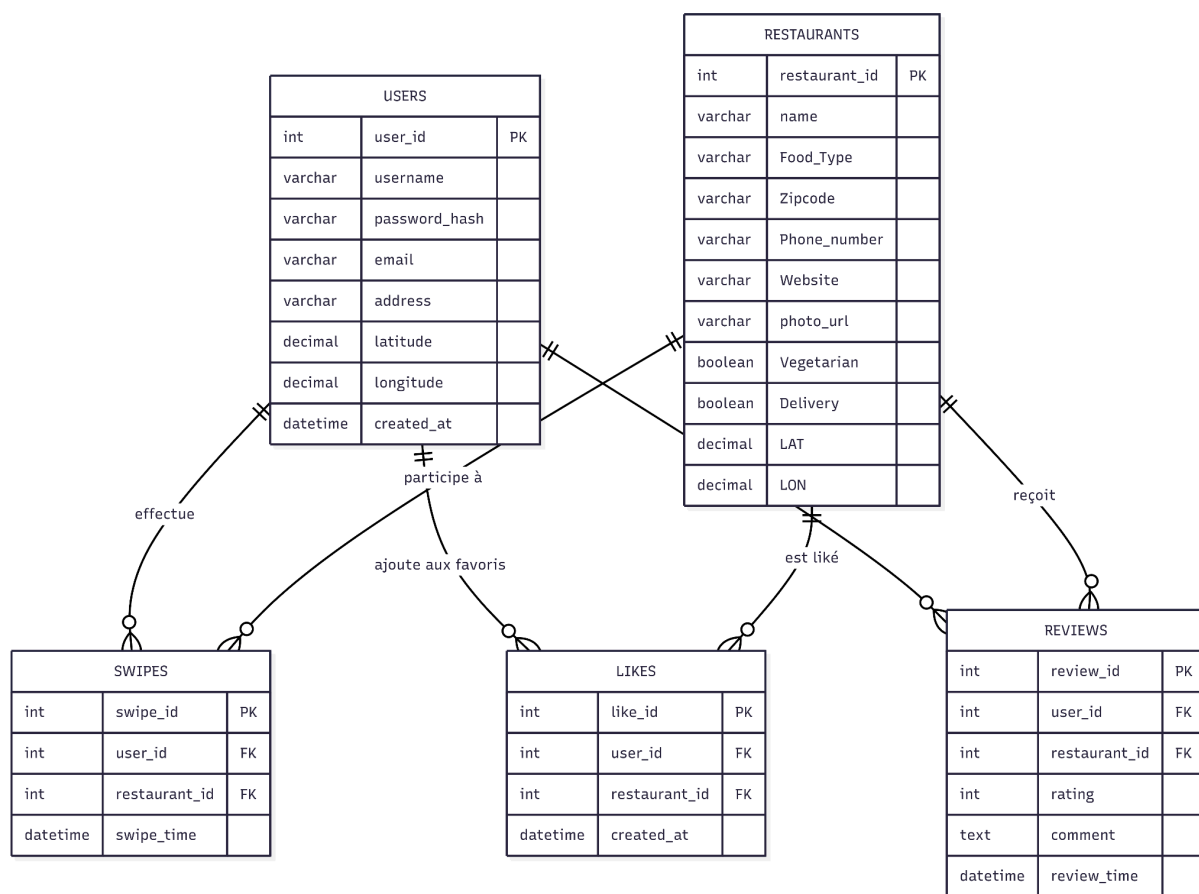


Figure [4] : Diagramme Entité-Association (ERD)

Un travail spécifique d'optimisation a été réalisé sur les clés primaires et étrangères pour assurer la fluidité des jointures SQL, notamment lors de la récupération de l'historique des "Likes" d'un utilisateur, qui nécessite de croiser instantanément les informations des tables restaurants et likes.

2. Développement de l'API (Node.js / Express) :

Le cœur applicatif de TinderFood est géré par un serveur développé en Node.js utilisant le framework Express. Ce choix technologique assure une grande flexibilité et permet de traiter les requêtes HTTP de manière asynchrone et non bloquante, ce qui est crucial pour maintenir la fluidité de l'application mobile.

L'API (située dans le fichier server.js) expose plusieurs points d'entrée (endpoints) standardisés qui permettent au client mobile d'interagir avec les données:

- **Authentification et Sécurité** : Les routes `/api/register` et `/api/login` gèrent l'accès utilisateur. Un point d'honneur a été mis sur la sécurité : les mots de passe ne sont jamais stockés en clair. Nous utilisons la bibliothèque `bcrypt` pour les hacher avant leur insertion en base de données, protégeant ainsi les utilisateurs même en cas de compromission du serveur.
- **Distribution des Données** : La route `/api/dishes` est sollicitée au lancement de l'application. Elle interroge la base de données pour récupérer la liste des restaurants, qui sont ensuite filtrés et triés par distance directement sur le client.
- **Gestion des Interactions** : Chaque action de l'utilisateur déclenche une requête spécifique :
 - `POST /api/swipe` : Enregistre un vote (duel gagné/perdu) dans la table `swipes`.
 - `POST /api/like` : Ajoute un restaurant aux favoris.
 - `POST /api/review` : Permet l'envoi d'une note et d'un commentaire.

Enfin, des *middlewares* techniques comme `cors` et `body-parser` ont été intégrés pour gérer les permissions d'accès (Cross-Origin Resource Sharing) et le traitement des corps de requêtes au format JSON.

Le diagramme de séquence ci-dessous illustre le cheminement d'une requête type, de l'action utilisateur jusqu'à la persistance en base de données :

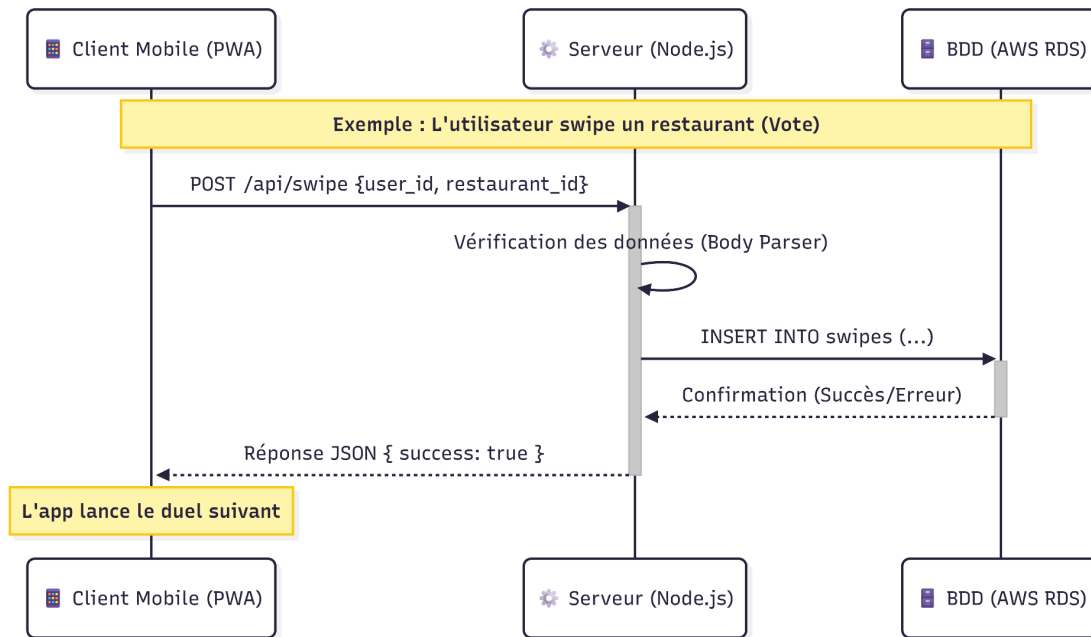


Figure [5] : Diagramme de séquence illustrant le traitement d'une requête de "Swipe" via l'API.

3. Qualité et Traitement de la Donnée :

Pour que le concept de "Duel visuel" fonctionne, la qualité des données présentées est aussi critique que le code lui-même. Une attention particulière a été portée à l'enrichissement et à la standardisation des informations des restaurants.

- **Enrichissement Automatisé (Pipeline Unsplash)** : Initialement, notre base de données contenait des informations textuelles mais manquait de visuels attrayants. Nous avons développé un script d'automatisation (`fill_db_with_api.js`) qui connecte notre base à l'API Unsplash. Ce script ne se contente pas de télécharger des images au hasard ; il utilise une table de correspondance sémantique (`KEYWORD_MAPPING`) pour associer intelligemment le type de cuisine à des termes de recherche visuels performants (ex: le type technique "japanese" déclenche une recherche "sushi", "fast_food" devient "junk food burger"). Cela garantit que chaque restaurant proposé dans l'interface possède une image de haute qualité correspondante, essentielle pour l'engagement utilisateur.
- **Standardisation et nettoyage** : Pour assurer le bon fonctionnement des filtres de recherche (développés côté mobile), nous avons normalisé les données en entrée. L'utilisation de types ENUM dans la base de données (pour les champs comme Vegetarian ou Take_away) force la cohérence des données. Cela évite les erreurs de saisie et permet à l'algorithme de tri de fonctionner sur des critères binaires stricts plutôt que sur du texte libre, améliorant ainsi la rapidité et la pertinence des résultats proposés lors des duels.

C. Front-end et Interface Mobile

1. Technologie utilisée (React Native / Expo) :

Pour le développement de l'interface mobile, notre choix s'est porté sur le framework React Native, couplé à la surcouche Expo. Ce choix technologique répond à plusieurs impératifs critiques du projet, à commencer par la nécessité d'une approche multiplateforme. En effet, React Native nous permet de maintenir une base de code unique écrite en JavaScript, capable d'être déployée simultanément sur les environnements Android et iOS. Cette stratégie unifiée réduit considérablement la charge de travail par rapport à un développement natif traditionnel qui aurait exigé la maîtrise de deux langages distincts, tels que Kotlin et Swift.

De plus, l'intégration de la suite d'outils Expo a joué un rôle déterminant dans notre phase de prototypage rapide. Cet écosystème a permis à l'équipe de visualiser les modifications en temps réel sur nos propres terminaux via l'application compagnon, éliminant ainsi la complexité de configuration des environnements de compilation lourds comme Android Studio ou Xcode. Cette agilité nous a permis d'itérer fréquemment sur le design et les fonctionnalités.

Enfin, sur le plan de la performance, cette stack technique se distingue des applications web classiques ou hybrides. Contrairement à une simple page web affichée dans un navigateur mobile, React Native instancie des composants graphiques natifs. Cela garantit une fluidité de défilement optimale et une réactivité tactile immédiate, des qualités indispensables pour la mécanique de "swipe" et de clic rapide que nous avons placée au cœur de l'expérience utilisateur.

2. Implémentation du "Duel" :

Le mécanisme de "Duel" constitue la fonctionnalité centrale de l'application, conçue pour transformer une décision fastidieuse en un jeu rapide. Pour garantir une fluidité absolue et éliminer toute latence perceptible entre deux choix, nous avons opté pour une stratégie de pré-chargement des données. Concrètement, au lancement de la session, l'application récupère un lot complet de restaurants pertinents depuis le serveur via une requête unique, plutôt que d'interroger l'API à chaque nouvelle carte affichée.

Choisis ton plat préféré 🍱



Puerto Inca
peruvian



Moshi Mo
japanese

J'ai choisi 🍷

Réinitialiser

Figure [6] : Interface d'un duel

L'algorithme de confrontation s'exécute ensuite intégralement côté client, selon une logique de "tournoi" à élimination directe. L'écran présente en permanence deux options : un "champion" (le restaurant retenu lors du vote précédent) face à un "challenger" (le nouveau restaurant proposé). Lorsqu'un utilisateur sélectionne une option par un clic, le restaurant écarté est instantanément retiré de la pile locale et remplacé par le candidat suivant dans la liste, tandis que le vainqueur reste à l'écran pour le duel suivant. Cette approche locale permet d'enchaîner les décisions instantanément, sans attendre de réponse serveur à chaque interaction.

Enfin, l'expérience est renforcée par des retours visuels dynamiques gérés par le moteur de rendu de React Native. Des animations de transition fluides ont été implémentées pour matérialiser le rejet ou l'acceptation, offrant un feedback immédiat à l'utilisateur. Ce système, couplé à l'absence de temps de chargement inter-duel, maintient un rythme soutenu qui favorise une prise de décision rapide et ludique, résolvant ainsi la problématique de l'indécision.

Voici la logique algorithmique simplifiée du duel :

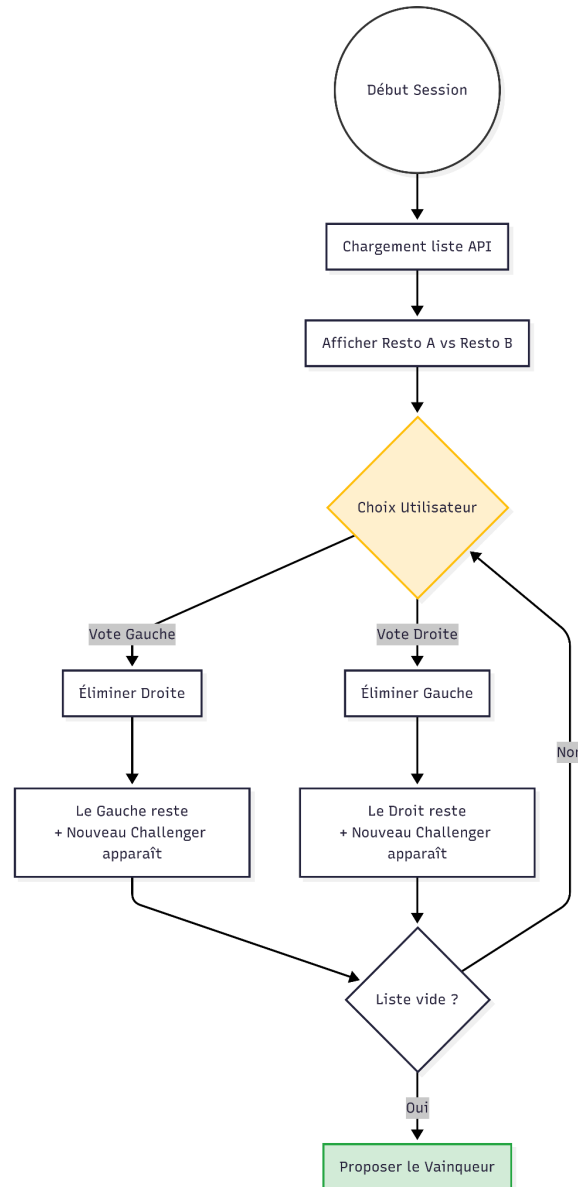


Figure [7] : Logigramme du fonctionnement de l'algorithme de duel "Winner Stays On"

3. UX/UI et Accessibilité :

L'interface a été conçue avec l'objectif premier de réduire la charge cognitive de l'utilisateur, souvent saturé d'informations sur les applications concurrentes. Nous avons pris le parti d'un design radicalement minimaliste où l'écran de jeu est épuré pour ne laisser place qu'à l'essentiel, à savoir les visuels des plats et les noms des restaurants. Cette suppression volontaire des menus complexes ou des détails superflus durant la phase de jeu permet de focaliser l'attention sur la décision binaire immédiate, accélérant ainsi le processus de choix sans générer de fatigue mentale.

En termes d'ergonomie, une attention particulière a été portée à la zone de confort du pouce. Sachant que l'application est vouée à être utilisée en situation de mobilité, souvent à une seule main, nous avons dimensionné et positionné les éléments

interactifs majeurs, comme les boutons de vote ou l'accès aux fiches détails, dans la partie inférieure de l'écran. Cette disposition évite à l'utilisateur de devoir modifier sa prise en main pour atteindre les fonctionnalités critiques. De plus, les zones tactiles ont été élargies pour tolérer les imprécisions du clic, rendant la navigation fluide et moins frustrante sur les petits écrans.

Enfin, l'accessibilité et l'engagement utilisateur ont été renforcés par un système de feedback visuel constant. L'application ne se contente pas d'afficher des données statiques ; elle réagit organiquement à chaque interaction. L'élimination d'un restaurant est matérialisée par une animation de sortie fluide, tandis que la validation du choix final déclenche une animation festive de type confettis. Ces retours visuels immédiats confirment la prise en compte de l'action par le système et améliorent l'expérience, apportant une gratification instantanée qui encourage l'utilisateur à finaliser son parcours de décision.

4. Fonctionnalités Clés intégrées :

Outre la mécanique ludique du jeu, l'utilité réelle de TinderFood repose sur sa capacité à ancrer la recherche dans le contexte immédiat de l'utilisateur. Pour ce faire, nous avons intégré un module de géolocalisation natif. Dès l'initialisation de la session, l'application sollicite l'autorisation d'accéder aux coordonnées GPS du terminal. Cette donnée n'est pas accessoire ; elle agit comme le filtre primaire de notre algorithme. En calculant la distance en temps réel entre la position de l'utilisateur et les coordonnées de chaque établissement stocké en base, le système trie les résultats par proximité, garantissant que les options proposées sont toujours pertinentes et physiquement accessibles.

La seconde fonctionnalité majeure concerne la concrétisation du choix, c'est-à-dire le passage de la décision virtuelle à l'action physique. Une fois le restaurant vainqueur désigné, l'application ne laisse pas l'utilisateur sans solution. Nous avons mis en place un système de liens profonds, ou deep linking, qui permet d'ouvrir automatiquement l'application de navigation installée sur le téléphone, qu'il s'agisse de Google Maps ou d'Apple Plans. Le lien généré contient déjà les paramètres de destination précis, initiant immédiatement le calcul d'itinéraire depuis la position actuelle. Cette intégration fluide supprime la nécessité de copier-coller l'adresse manuellement, réduisant les frictions jusqu'au dernier moment de l'expérience.

D. Défis Techniques et Solutions

Le développement de TinderFood a mis l'équipe face à plusieurs problématiques techniques inhérentes à la nature mobile et instantanée du projet. Le premier obstacle majeur a été la gestion du démarrage à froid. Dans un système de recommandation classique, l'algorithme a besoin de l'historique de l'utilisateur pour

proposer des contenus pertinents. Or, un nouvel inscrit sur notre application ne possède aucune donnée comportementale. Pour contourner ce problème et éviter de proposer des restaurants aléatoires peu pertinents, nous avons exploité la géolocalisation comme pivot central. En triant par défaut les résultats par pure proximité géographique lors de la première session, nous garantissons une utilité immédiate à l'application dès la première seconde d'utilisation, le temps que le système accumule suffisamment de données de "Likes" pour affiner les propositions futures.

Un second défi technique concernait la latence réseau et la fluidité de l'expérience utilisateur. La mécanique de jeu repose sur une prise de décision rapide ; attendre un chargement serveur entre chaque duel aurait brisé le rythme et l'immersion ("Flow"). Pour résoudre cela, nous avons opté pour une stratégie de chargement par lots (batching). Au lieu d'effectuer une requête HTTP à chaque interaction, l'application pré-charge une liste d'une vingtaine de restaurants au démarrage. Les interactions de vote et d'élimination sont ensuite gérées intégralement par le processeur du téléphone en manipulant le tableau de données local. Cette approche décorrèle l'interface utilisateur des aléas de la connexion internet, assurant des animations fluides à 60 images par seconde, même en cas de réseau instable.

Enfin, nous avons dû répondre à l'exigence de qualité visuelle des données. Une application basée sur le choix par l'image ne peut se permettre d'afficher des photos pixélisées ou manquantes, ce qui était le cas avec nos données brutes initiales. La solution a résidé dans l'automatisation de l'enrichissement des données via un script Node.js dédié. En couplant notre base de données à l'API Unsplash et en créant un dictionnaire de correspondance sémantique précis, nous avons pu attribuer automatiquement des visuels haute définition adaptés à chaque type de cuisine. Cela a permis de transformer une base de données textuelle aride en un catalogue visuel attrayant sans nécessiter de saisie manuelle fastidieuse, garantissant une esthétique professionnelle uniforme sur l'ensemble de l'application.

IV. Conclusion

A. Perspectives

Avec une couche de ML, nous pourrions faire évoluer TinderFood d'un tri "général" à une recherche vraiment personnelle. Concrètement, l'application apprend de l'historique (ce qu'on garde, ce qu'on écarte, nos avis) et reconnaît nos goûts pour proposer dès le départ des restaurants plus proches de l'envie du moment. Elle s'adapte en direct à nos choix, évite de répéter des options peu pertinentes, et glisse parfois une découverte cohérente. Résultat : moins de duels nécessaires, un match

plus rapide, des suggestions plus justes et une satisfaction plus élevée après la visite.

Concernant les données, nous avons identifié la nécessité d'élargir et fiabiliser la base. Concrètement, on ajouterait les fourchettes de prix, des horaires à jour, et des tags utiles (ambiance, terrasse, options végétariennes/halal, "rapide le midi"), ainsi que des menus et photos plus propres. On commencerait par un pipeline simple (sources ouvertes/partenaires) avec dé-doublonnage et normalisation des cuisines. Cela débloque des filtres plus précis ("ouvert maintenant", "budget") et des résultats plus cohérents.

Côté expérience, on veut aller vers un ressenti plus "app". On ajouterait des gestes natifs avec retour haptique, des micro-interactions plus nettes (zoom, blur, "shake"), un mode hors-ligne partiel (dernier duel en cache) et une accessibilité renforcée (tailles adaptatives, alternatives clavier/lecture d'écran). On aimerait aussi un mode groupe très simple (lien de session + vote) et un cold-start plus malin avec quelques duels diversifiés pour cerner vite les goûts.

Pour transformer plus facilement le match en sortie, on améliorerait l'intégration Maps (mode de transport, ETA), on ajouterait des raccourcis d'appel et des liens de réservation quand c'est disponible, ainsi qu'une shortlist top 3 partageable (WhatsApp/Message). Après la visite, on enverrait un rappel doux pour l'avis, utile à la fois pour l'utilisateur et pour la qualité de nos futures recommandations.

B. Intérêt professionnel

Ce projet a constitué une véritable immersion dans la réalité d'une équipe produit. Il nous a contraints à dépasser la simple exécution scolaire pour adopter une posture professionnelle : cadrer le besoin, prioriser les fonctionnalités critiques selon la méthode MoSCoW et respecter des délais stricts.

Sur le plan technique, nous avons consolidé des compétences recherchées sur le marché :

- Architecture Cloud : Le déploiement et la gestion d'une base de données relationnelle sur AWS RDS.
- Développement Fullstack : La conception d'une API sécurisée (Node.js) et son interaction avec une interface mobile fluide (React Native).
- Data Engineering : La mise en place de pipelines d'enrichissement de données (API Unsplash) et la normalisation des entrées.

Enfin, nous avons intégré des réflexes indispensables en entreprise tels que la sécurisation des accès (variables d'environnement), le souci de la qualité des

données et la documentation des choix techniques. TinderFood constitue désormais un actif concret et démontrable qui enrichit notre portfolio et crédibilise notre profil auprès de futurs recruteurs.

C. Bilan sur le travail d'équipe

La dynamique de groupe a été l'un des facteurs clés de la réussite de ce projet. La répartition des tâches, alignée sur les majeures de chacun (Systèmes Embarqués, Data, Énergie), a permis une grande efficacité opérationnelle tout en favorisant l'entraide sur les points bloquants. Nous avons su tenir les délais et livrer un MVP fonctionnel grâce à une coordination fluide.

Cependant, cette expérience nous a aussi permis d'identifier des axes d'amélioration. Nous avons notamment constaté qu'une synchronisation plus précoce entre les maquettes de design (Front-end) et les contraintes de la structure de données (Back-end) aurait permis d'éviter certains allers-retours en phase de développement. De même, intégrer des tests utilisateurs plus tôt dans le cycle aurait pu valider certaines hypothèses ergonomiques plus rapidement.

Malgré ces défis, nous avons gagné en rigueur dans notre communication et nos arbitrages. Cette expérience nous a appris à coordonner des profils techniques hétérogènes autour d'un objectif commun, une compétence essentielle pour nos futures carrières d'ingénieurs.